MODULE 13 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Stages, file formats and the load path

Domain 3 begins. The stage symbols are free marks, and candidates routinely guess at them.

DOMAIN 3 of 5 WEIGHT 18% MODULE 13 of 31

admin@sthithapragna.com

Loading is always **two steps**. Get the files to a stage. Then **COPY INTO** a table.

The four stage types

STAGE	HOW YOU REFERENCE IT	WHO IT BELONGS TO
User	@~	One user. Allocated automatically. Nobody else can see it.
Table	@%mytable	One table. Allocated automatically. Same name as the table.
Named internal	@mystage	A database object. Shared by whoever has privileges on it.
Named external	@myextstage	Points at S3, Azure Blob or Google Cloud Storage.

ON THE EXAM

Learn the three symbols cold. Tilde is the user, percent is the table, and a bare name is a named stage.

Table stages have real restrictions

- **An implicit stage tied to the table. Not a separate database object.** So it has **no grantable privileges of its own**.
- To stage, list, query or drop files on it you must **be the table owner**.
- **Cannot be altered or dropped.** It exists because the table exists.
- **Does not support transforming data while loading.** You cannot use a query as the source for COPY.
- **Iceberg tables do not support table stages** at all.

> ON THE EXAM
>
> Every restriction follows from one fact: it is not an object, so there is nothing to grant on and nothing to alter.

When the automatic stages are not enough

USER AND TABLE STAGES

Convenient, and narrow

- Allocated automatically, nothing to create
- User stage: only that user, ever
- Table stage: only the table owner, one table
- Neither can be granted, altered or dropped

NAMED INTERNAL STAGES

The flexible option

- A real database object with privileges
- Multiple users can load from it
- Data can go to multiple tables
- Supports transformation during load

THE TIEBREAKER

Two questions force a named stage: does more than one person need it, and does more than one table need it?

External stages and storage integrations

- A named external stage points at a location in **S3, Azure Blob Storage or Google Cloud Storage**.
- A **storage integration** holds the identity and access configuration, so credentials never appear in the stage definition.
- The integration is created once by an administrator and reused by many stages. Credentials are never seen by the people creating stages.
- Embedding cloud credentials directly in CREATE STAGE is possible in some cases and is **the wrong answer** on a security question.

> ON THE EXAM
>
> Storage integration is one of four integration types. Storage for buckets, API for HTTPS, notification for messaging, catalog for Iceberg.

Directory tables and stage encryption

- A **directory table** sits over a stage and holds file-level metadata, so files can be listed and queried in SQL.
- It is not a separate object. It is a **property of the stage**, enabled with DIRECTORY = (ENABLE = TRUE).
- It **requires a warehouse and therefore costs money**, which is why it is optional rather than automatic.
- **Internal stages are always encrypted by Snowflake.** For external stages, you manage encryption and compression.

> ON THE EXAM
>
> Directory tables are how unstructured data becomes queryable: file names, sizes and URLs, addressable in SQL.

File formats Snowflake loads

FORMAT	KIND
CSV, and delimited text generally	Structured
JSON	Semi-structured
Avro	Semi-structured
ORC	Semi-structured, columnar
Parquet	Semi-structured, columnar
XML	Semi-structured

ON THE EXAM

Six formats. One structured, five semi-structured. That split is the answer to a surprising number of questions.

File format objects and compression

- A **file format object** stores the settings once: type, delimiter, header handling, null handling, date formats.
- Reference it from a stage or from a COPY statement, instead of repeating options on every load.
- Supported compression: **GZIP, BZ2, BROTLI, ZSTD, SNAPPY, DEFLATE and RAW_DEFLATE**.
- Snowflake **decompresses automatically**. You do not uncompress files before loading them.

> ON THE EXAM
>
> A file format object is the same pattern as a policy: define once, reference many times, change in one place.

File sizing, and the sentence that explains it

- **The number of parallel load operations cannot exceed the number of data files.** Everything else follows from this.
- Aim for roughly **100 to 250 MB compressed**, or larger. This applies to bulk loads *and* to Snowpipe.
- **Very large files, 100 GB or more, are not recommended.** One file cannot be split across threads.
- Files under about **10 MB** should be aggregated. Per-file overhead starts to dominate.
- Use a **path or prefix** so COPY lists only the relevant files rather than traversing a whole bucket.

> ON THE EXAM
>
> One 200 GB file on a 2X-Large loads no faster than on an X-Small. There is nothing for the extra threads to do.

Which stage for which job

THE SITUATION	THE STAGE
A quick personal load nobody else needs	User stage, @~
One table, loaded by its owner, no transformation	Table stage, @%mytable
Several people, several tables, or transformation	Named internal stage
The data already lives in your own cloud storage	External stage plus a storage integration
Files must be listed and queried in SQL	A directory table on the stage

ON THE EXAM

Row three is the default in real accounts. The automatic stages run out of room quickly.

A team receives a nightly set of CSV files containing order data. The same files must be loaded into three tables: a raw table, a cleaned table where several columns are cast and reordered during load, and an archive table.

Four engineers run the pipeline on rotation. An engineer proposes staging the files on the raw table's table stage.

What is wrong with the proposal?

A Table stages cannot serve multiple tables, cannot be granted to other users, and do not support transformation during load.

B Table stages only accept a single file per load, so the nightly set would need combining first.

C Nothing is wrong, provided each engineer is granted USAGE on the table stage.

D Table stages cannot hold CSV files. Only named stages support structured formats.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A Three restrictions, all of which this scenario hits.

Multiple tables, multiple users, and transformation during load. Every one of those is something a table stage cannot do.

B One file per load.

Invented. Any stage type holds many files, and loading many at once is exactly what you want.

C Grant USAGE to each engineer.

A table stage is not a database object and has no grantable privileges. There is nothing to grant.

D Table stages cannot hold CSV.

All stage types hold any supported format. Format is a property of the file and the COPY, not the stage.

THE SIGNAL

Three independent blockers in one scenario. Any one alone would force a named internal stage.

A nightly bulk load reads a single compressed file of about 180 GB from an external stage. It takes just over five hours on a Large warehouse.

The team resizes to 2X-Large. Runtime is essentially unchanged. Query Profile shows no spilling and no queueing, and the warehouse appears largely idle during the load.

Why did resizing not help?

A COPY INTO ignores warehouse size and always runs on a fixed serverless pool.

B The file must be decompressed before loading, and decompression is single-threaded regardless of size.

C Parallel load operations cannot exceed the number of files. One file means one thread, whatever the size of the warehouse.

D External stages cap load parallelism at four threads. An internal stage would have scaled.

◆ PAUSE AND DECIDE

Why did resizing not help?

A. COPY runs on a serverless pool.
COPY INTO runs on the warehouse you name. Snowpipe is the serverless loader, and it is a different thing.

B. Decompression is single-threaded.
Sounds plausible and is not the documented reason. Snowflake decompresses automatically, and the constraint is file count.

C. Parallelism cannot exceed the number of files.
Documented directly. One file gives one load operation, so the extra threads on a 2X-Large have nothing to do.

D. External stages cap at four threads.
Invented. Stage type does not limit parallelism; the number of files does.

THE SIGNAL

Split the file. Roughly a thousand files of 180 MB would saturate the warehouse and cost less than the 2X-Large did.

> A security review finds that a CREATE STAGE statement in a version-controlled deployment script contains an AWS access key and secret key in plain text. Several analysts have been granted the privilege to create stages.
>
> The team needs Snowflake to keep reading the same S3 bucket without exposing credentials to anyone who can read the script or run SHOW STAGES.

What should be implemented?

A. Move the credentials into a Snowflake SECRET object and reference it from the stage.

B. Create a storage integration holding the IAM configuration, and reference it from the external stage.

C. Copy the files into a named internal stage, which removes the need for cloud credentials.

D. Apply a masking policy to the stage definition so the keys are hidden from SHOW STAGES.

◆ PAUSE AND DECIDE

What should be implemented?

A. A SECRET object referenced from the stage.

Secrets serve API integrations and external access, notably Git. Cloud storage authentication is what storage integrations are for.

B. A storage integration referenced from the external stage.

Exactly its purpose. An administrator creates it once, stage creators never see credentials, and rotation happens in one place.

C. Copy files into an internal stage.

Something must still read the bucket to copy them, needing the same credentials. It also duplicates storage.

D. Mask the stage definition.

Masking policies protect column values in tables and views. They do not apply to object definitions.

THE SIGNAL

Four integration types, and picking the wrong one is the commonest error here. Storage for buckets, API for HTTPS endpoints.

A team stores several hundred thousand scanned invoices as PDFs in an external stage. They want to build a table listing every file with its name, size and last-modified time, join that to an internal register, and generate URLs so an application can retrieve individual documents.

An engineer proposes running LIST on the stage each morning and parsing the output into a table.

What is the better approach?

A Load the PDFs into a VARIANT column, which makes their metadata queryable in SQL.

B Create an external table over the stage, which exposes file metadata as columns.

C Nothing better exists. LIST is the only way to see stage contents, so parsing its output is correct.

D Enable a directory table on the stage, which exposes file metadata and URLs as queryable columns.

◆ PAUSE AND DECIDE

What is the better approach?

A. Load the PDFs into a VARIANT column.

VARIANT holds semi-structured data and is capped at 16 MB per row. A PDF is unstructured; this is the wrong container.

B. An external table over the stage.

External tables read the contents of structured and semi-structured files. They do not catalogue unstructured documents.

C. LIST is the only option.

Directory tables exist for exactly this. Parsing command output into a table is the workaround they replace.

D. Enable a directory table on the stage.

Purpose-built: file names, sizes, last-modified times and URLs as queryable columns, joinable like any table.

THE SIGNAL

Directory tables need a warehouse and so cost money. That is the trade, and it is why they are not on by default.

What to carry into the next module

- **@~ is the user stage, @%table is the table stage, @name is a named stage.** Free marks.
- **Table stages are not objects.** No grantable privileges, no ALTER or DROP, one table, no transformation on load.
- **Parallel loads cannot exceed the number of files.** One huge file gets one thread. Aim for 100 to 250 MB compressed.
- **Storage integration** for external stages, so credentials never appear in a stage definition.
- **Directory tables** make stage contents queryable. They need a warehouse, so they cost money.

> ON THE EXAM
>
> Next: COPY INTO itself. Error handling, load metadata, transformation on load, and unloading back out.